8. Introduction

Most people meet this kind of problem at the grocery store before they ever meet it in an interview. You need a few things from two different aisles, and the store is running a combo deal that covers one item from each aisle at a lower combined price. Do you take the deal, and if so, how much of it?

LeetCode 3789, *Minimum Cost to Acquire Required Items*, turns that everyday question into a clean algorithmic puzzle. On the surface it looks like a simple cost-comparison problem — buy the cheap thing, skip the expensive thing. But the moment you try to write the "obvious" greedy rule, it quietly produces the wrong answer on the problem's own sample input. That gap between what feels right and what's actually correct is exactly what interviewers are probing for.

This problem is worth learning for three reasons. First, it's a great lesson in why "greedy that sounds right" is not the same as "greedy that is right" — you have to prove the greedy rule, not just trust your gut. Second, it's a strong example of a recurring technique: when a search space is too large to brute-force, look for a small, provably sufficient set of candidate answers instead of scanning everything. Third, the exact same shape — single-purpose prices plus a combined discount — shows up constantly in real systems: checkout bundle deals, cloud pricing tiers, coupon stacking, bulk discounts. Once you've internalized this pattern, you'll start recognizing it everywhere.

9. Problem Overview

Here's the problem in plain terms, no transcript copy-paste.

You have two separate requirements — call them **requirement 1** and **requirement 2** — and you need to acquire at least `need1` units toward the first and at least `need2` units toward the second.

There are three ways to acquire units:

- **Type 1 item**: costs `cost1` each, counts only toward requirement 1.
- **Type 2 item**: costs `cost2` each, counts only toward requirement 2.
- **Combined item**: costs `costBoth` each, counts toward *both* requirements simultaneously — one combined item reduces what's still owed on requirement 1 by one, and what's still owed on requirement 2 by one, at the same time.

You want the cheapest total spend that satisfies both requirements. You can buy any non-negative integer quantity of each type, and you're allowed to overbuy — buying more combined items than you

strictly need for one requirement is fine if it's still cheaper overall.

Constraints keep the individual prices small (up to 1,000,000) but let the requirements themselves run enormous — up to 1,000,000,000. That last detail is the whole game: you cannot loop over "how many combined items should I buy" one at a time, because that loop alone could run a billion iterations. Whatever algorithm you use has to be independent of how large `need1` and `need2` are — effectively constant time.

10. Example

Let's use the problem's own numbers:

- `cost1 = 3`, `cost2 = 2`, `costBoth = 1`
- `need1 = 3`, `need2 = 2`

Option A — buy only single-type items. 3 type-1 items (3 × 3 = 9) + 2 type-2 items (2 × 2 = 4) = **13**. Expensive, ignores the discount entirely.

Option B — buy combined items only up to the smaller requirement, then top up. `need2 = 2` is smaller, so buy 2 combined items (2 × 1 = 2), covering requirement 2 fully and knocking requirement 1 down to 1 remaining. Top up with 1 type-1 item (1 × 3 = 3). Total: 2 + 3 = **5**.

Option C — buy combined items all the way up to the larger requirement. `need1 = 3` is larger, so buy 3 combined items (3 × 1 = 3). Requirement 1 is fully covered (3 of 3). Requirement 2 needed only 2, but you've bought 3 — that's one "wasted" combined unit, except it isn't wasted, because you never paid for a type-2 item at all. Total: 3 × 1 = **3**.

Option C wins. Every unit of the overbuy (3 → 2) still beat paying `cost2` for a type-2 item, because `costBoth (1) < cost2 (2)`. That's the whole trap in one example: stopping at the smaller requirement looks like "efficient use of the discount," but it's actually leaving cheaper combined units on the table.

11. Intuition

Start from the instinct almost everyone reaches for first: buy combined items only for the *overlap* between the two requirements — that is, `min(need1, need2)` — then fill in whatever's left of the larger requirement with single-type purchases. It feels tidy: you're using the combo deal exactly as far as it applies to both sides, and nothing more.

The example above shows this instinct is wrong, and it's wrong for a specific, generalizable reason: a combined item's value doesn't vanish the moment one requirement is satisfied. It only stops being useful when its price is no longer competitive with buying the remaining need the "normal" way. You keep buying combined items past the smaller requirement precisely when `costBoth` is still cheaper than `cost1` (or `cost2`) per unit — you're just not "using" the half of that unit that goes toward an already-satisfied requirement. You're not paying extra for it; you're simply not benefiting from it on that side, which is fine, because you weren't going to pay for a separate item on that side anyway.

So the real question isn't "how much overlap is there" — it's "at what point does buying one more combined item stop being a good trade against the alternative?" There are exactly two points where this trade-off could flip: the moment you've covered `need2` (the smaller requirement, where you'd otherwise switch to type-1 purchases) and the moment you've covered `need1` (the larger requirement, where you'd otherwise switch to type-2 purchases, or where buying combined items simply stops making sense because both requirements are already met).

That gives you three meaningful quantities of combined items to ever try buying: **zero** (don't use the deal at all), **the smaller requirement**, and **the larger requirement**. Nothing between these points can be better than the best of its neighbors, because the cost function is piecewise linear and only changes slope at these breakpoints — a fact we'll lean on rather than prove formally here, but it's the same idea behind why you only check the corners of a feasible region in linear optimization.

12. Brute Force Solution

Idea: Try every possible quantity of combined items from `0` up to `max(need1, need2)`, price each one out, and keep the cheapest.

Algorithm: 1. For `x` from `0` to `max(need1, need2)`: - Remaining type-1 units needed: `max(0, need1 - x)` - Remaining type-2 units needed: `max(0, need2 - x)` - Total cost: `x * costBoth + remaining1 * cost1 + remaining2 * cost2` 2. Return the minimum total cost found.

Advantages: Trivially correct — it checks literally every option, so there's no risk of missing the optimum. Easy to reason about and easy to verify against.

Disadvantages: `need1` and `need2` can be up to 1,000,000,000. A loop of a billion iterations will not finish in any reasonable time limit. This approach is correct but unusable at the given constraints — it exists only as a mental baseline and a correctness oracle for testing the fast solution on small inputs.

Complexity: Time `O(max(need1, need2))`, space `O(1)`. The time bound is the disqualifying factor.

13. Optimal Solution

The key realization from the intuition section is that the cost function, as a function of `x` (how many combined items you buy), is **piecewise linear** with at most two breakpoints. Here's why.

Define `cost(x) = x * costBoth + max(0, need1 - x) * cost1 + max(0, need2 - x) * cost2`.

As `x` increases from 0: - While `x < min(need1, need2)`, every extra combined item removes one unit of *both* remaining requirements. The slope of `cost(x)` here is `costBoth - cost1 - cost2`. - Once `x` passes the smaller requirement (say `need2`), that side is saturated — extra combined items no longer reduce `max(0, need2 - x)` below zero. Now only the larger requirement's remaining amount shrinks. The slope changes to `costBoth - cost1` (if `need1` is larger) or `costBoth - cost2` (if `need2` is larger). - Once `x` passes the larger requirement too, both remaining terms are pinned at zero, and every extra combined item is pure waste. The slope becomes `costBoth`, which is always positive — so buying beyond the larger requirement is never worth it.

Region	Which requirements still "active"	Slope of cost(x)
`0 ≤ x ≤ min(need1, need2)`	both	`costBoth - cost1 - cost2`
`min(need1,need2) < x ≤ max(need1,need2)`	larger only	`costBoth - cost1` or `costBoth - cost2`
`x > max(need1, need2)`	none	`costBoth` (always rising)

A piecewise-linear function's minimum over an interval always occurs either at the interval's boundary, or at one of the points where the slope changes. That's exactly `x = 0`, `x = min(need1, need2)`, and `x = max(need1, need2)`. Nothing past the larger requirement can help, since the slope there is strictly positive. So we only ever need to evaluate `cost(x)` at three values of `x`: `0`, `min(need1, need2)`, and `max(need1, need2)`.

That's the entire trick: three candidates, one shared pricing formula, no dependence on how large the requirements get.

14. Python Code

```
def min_cost_to_acquire(cost1: int, cost2: int, cost_both: int,
                        need1: int, need2: int) -> int:
    """Minimum cost to satisfy both requirements using single-type
    items and a combined item, by checking only the three candidate
    quantities of combined items where the cost function can bend.
```

```
    """

    def price(num_combined: int) -> int:
        remaining1 = max(0, need1 - num_combined)
        remaining2 = max(0, need2 - num_combined)
        return num_combined * cost_both + remaining1 * cost1 + remaining2 * cost2

    smaller_need = min(need1, need2)
    larger_need = max(need1, need2)

    candidates = (0, smaller_need, larger_need)
    return min(price(x) for x in candidates)
```

15. Code Walkthrough

- `price(num_combined)` is the shared pricing formula: given a chosen number of combined items, it computes what's still owed on each requirement (`max(0, need - num_combined)` so a satisfied requirement never goes negative), and adds up the total spend across all three item types.
- `smaller_need` / `larger_need` pin down the two breakpoints identified in the optimal-solution analysis — the two points where the cost function's slope can change.
- `candidates = (0, smaller_need, larger_need)` is the entire search space. No loop over the actual requirement sizes, no recursion, no DP table — just three fixed points.
- `min(price(x) for x in candidates)` prices all three and keeps the cheapest. Each call to `price` is `O(1)`, so the whole function runs in constant time regardless of how large `need1` and `need2` are.

16. Dry Run

Using `cost1 = 3`, `cost2 = 2`, `cost_both = 1`, `need1 = 3`, `need2 = 2`:

- `smaller_need = 2`, `larger_need = 3`
- `price(0)`: remaining1 = 3, remaining2 = 2 → `0*1 + 3*3 + 2*2 = 0 + 9 + 4 = 13`
- `price(2)`: remaining1 = 1, remaining2 = 0 → `2*1 + 1*3 + 0*2 = 2 + 3 + 0 = 5`
- `price(3)`: remaining1 = 0, remaining2 = 0 → `3*1 + 0*3 + 0*2 = 3 + 0 + 0 = 3`

`min(13, 5, 3) = 3` — matching the expected answer, and confirming that the "buy up to the larger requirement" candidate is the one that actually wins here.

17. Complexity Analysis

Time: `O(1)`. The function evaluates `price()` exactly three times, and each evaluation does a fixed number of arithmetic operations — no loop that scales with `need1`, `need2`, or any of the costs.

Space: `O(1)`. Only a handful of integer variables are held at once; no auxiliary data structures grow with input size.

Why this is correct, not just fast: correctness rests on the piecewise-linear argument from the optimal-solution section — the cost function's slope can only change at `min(need1, need2)` and `max(need1, need2)`, and is strictly increasing past the larger requirement. A piecewise-linear function restricted to `x ≥ 0` attains its minimum at one of its slope-change points or at its left boundary (`x = 0`), which is precisely the three candidates checked. Nothing outside that set can beat the best of those three.

18. Alternative Solutions

A ternary-search approach is sometimes suggested for "minimize a convex-looking function over a range" problems, but it doesn't apply cleanly here: the cost function is piecewise linear, not strictly convex in the way ternary search needs (it has flat or near-flat stretches depending on the cost relationships), and it would add `O(log(max(need1, need2)))` complexity for no benefit over simply checking the three provably sufficient points directly. Binary search on the answer value is also unnecessary — there's no monotonic "is this total achievable" predicate to search over; the problem is a direct cost minimization, not a feasibility threshold. The three-candidate check is strictly simpler and strictly faster than either alternative, which is why it's the solution worth learning rather than a shortcut to memorize.

19. Edge Cases

- **`costBoth` more expensive than buying separately** (`costBoth >= cost1 + cost2`): the combined item is never worth buying at all. `price(0)` will correctly win since the other two candidates only add cost without benefit.
- **One requirement is zero**: say `need2 = 0`. Then `smaller_need = 0`, so `price(0)` and `price(smaller_need)` are identical, and the real contest is between buying nothing combined and buying `need1` combined items. The formula handles this without special-casing because `max(0, 0 - x)` is always zero.
- **Both requirements equal** (`need1 == need2`): `smaller_need == larger_need`, so two of the three candidates collapse into the same value — harmless, just a redundant evaluation.
- **Maximum requirement values** (`need1 = need2 = 1,000,000,000`): the arithmetic still fits comfortably in Python's arbitrary-precision integers, and the algorithm's runtime is

unaffected since it never loops over the requirement magnitude.

- `costBoth` **equal to zero**: buying unlimited combined items is free, so both breakpoints (especially `larger_need`) will dominate and the answer should come out to `larger_need * 0 = 0` plus nothing else — correctly reflecting that the requirements are satisfied for free.

20. Common Mistakes

1. **Capping combined purchases at `min(need1, need2)` and never checking further.** This is the exact trap the example demonstrates — it silently produces a suboptimal (but plausible-looking) answer.
2. **Forgetting to clamp remaining need at zero.** Without `max(0, need - x)`, overbuying combined items can make the "remaining" count negative, which would incorrectly subtract cost instead of just contributing zero.
3. **Looping over every possible `x` from 0 to `max(need1, need2)`.** This passes on small test cases and silently times out or is simply infeasible at the real constraints (up to a billion).
4. **Only checking two candidates (`0` and `min(need1, need2)`) instead of three.** This misses the case where buying all the way up to the larger requirement wins, which is precisely the scenario in the worked example.
5. **Mixing up which requirement is "smaller" vs. "larger" when the two needs are compared**, leading to evaluating the wrong breakpoint or evaluating the same point twice while skipping the other.
6. **Assuming the combined item is always better (or always worse) without checking the actual numbers.** Whether buying past the smaller requirement helps depends entirely on how `costBoth` compares to `cost1` and `cost2` — there's no universal rule, which is exactly why all three candidates must be priced out rather than reasoned about qualitatively.

21. Interview Questions

- "Why is it safe to only check three values of `x` instead of searching the whole range?" — Expect a piecewise-linear / breakpoint argument, not just "I tried it and it worked."
- "What would change if there were three requirements instead of two, with combined items covering different pairs?" — Tests whether the candidate can generalize the breakpoint intuition rather than having memorized a fixed three-candidate formula.
- "How would you prove formally that checking the slope-change points is sufficient?" — A chance to walk through convexity/piecewise-linearity reasoning explicitly.
- "What's the time and space complexity, and why doesn't it depend on `need1` or `need2`?" — Checks that the candidate understands *why* the solution is constant time, not just that it is.

- "If prices could be negative (as a modeling trick, e.g., a rebate), would this approach still work?" — Probes whether they understand the assumptions baked into the slope analysis.

22. Similar Problems

- **LeetCode 2558 – Take Gifts From the Richest Pile** — different mechanics, but shares the "don't simulate every step when the numbers are huge" lesson via a heap instead of a closed-form formula.
- **LeetCode 1898 – Maximum Number of Removable Characters** — uses binary search on a monotonic feasibility function, a close cousin to the "find the right breakpoint instead of scanning" theme here.
- **LeetCode 2731 – Movement of Robots** — another problem where brute simulation is impossible at scale and a closed-form/sorted-pairs trick replaces it.
- **LeetCode 1055 – Shortest Way to Form String** — not pricing-related, but shares the pattern of reducing a huge search space to a small number of meaningful candidate checkpoints.
- **Classic "buy one get one" / tiered pricing problems** on other platforms — directly reuse the piecewise-linear breakpoint technique from this problem.

23. Key Takeaways

The headline lesson isn't really about pencils, notebooks, or LeetCode 3789 specifically — it's about recognizing when a cost function is piecewise linear and exploiting that structure instead of brute-forcing. The "obvious" greedy cap at the smaller requirement feels intuitive but is provably incomplete; the fix is to identify every point where the function's behavior can change — here, exactly two breakpoints plus the starting point — and check only those. That turns an algorithm that would need up to a billion iterations into one that runs in constant time with three arithmetic evaluations. This same move — find the handful of breakpoints, ignore everything between them — is a technique worth keeping in your back pocket for any pricing, tiering, or threshold-based optimization problem you'll meet again.

24. Watch the Video

If you'd rather see this traced out visually — the real example, the greedy instinct breaking live, and the three-candidate fix built up step by step — the full walkthrough is embedded here: {LINK}

25. About the Series

This breakdown is part of **Fun with Learning Technology**, a series dedicated to making coding interview problems actually make sense — not just memorized, but understood from first principles. Every episode walks through a real problem, shows where the natural-but-wrong instinct breaks, and builds up to the correct, provable solution the way an experienced engineer would actually discover it at a whiteboard.

26. Call To Action

If this walkthrough helped the trick click, give the video a like on YouTube and subscribe for more problems broken down the same way — intuition first, code second. Subscribe to the Substack newsletter for article versions like this one delivered straight to your inbox, drop a comment with the problem you want covered next, and share this with anyone prepping for interviews who still thinks greedy "feels right" is the same as greedy "is right."

Quick Review

Minimum cost with discounts on required items — what pattern?

Greedy/DP over cost-reduction choices: when buying X unlocks a discount on Y, think knapsack-style decision per item, not pure greedy.

Time complexity for this min-cost acquisition problem?

O(n log n) or O(n*k) depending on how many discount pairs you check per item — sort + pass, not brute force O(2^n).

Space complexity typically used here?

O(n) for tracking per-item best cost or discount eligibility; O(1) extra if done in-place on sorted array.

Trickiest bug when solving this problem?

Assuming the greedy 'buy cheapest first' always works — breaks when a discount depends on buying a MORE expensive item first.

Why does the first greedy instinct break here?

Picking lowest-cost items early can forfeit a discount that only triggers after buying a specific pricier required item.

Real trick to solve 'Minimum Cost to Acquire Required Items'?

Evaluate cost per item as min(buy directly, buy via qualifying purchase that unlocks discount) — compare both paths, don't assume one wins.
